AI Engineering and Claude Architect · Part IV, Production

AI Production Engineering, Lesson 2

AI Gateways and Model Routing

Everyone on the team had the provider key, and nobody knew which feature spent $40,000 last month. A gateway fixes that, and it also becomes the thing that can take every product down at once.

Module	AI10 of 13
Lesson	2 of 7
Prerequisites	AI10 L1, AI03 L4, AI02 L7, HLD M04 (gateways)
Exam relevance	Out of scope as a topic. Model choice appears in TS 1.3 and 4.5 contexts
Reading time	~30 minutes

Contents

1. AI Gateways and Model Routing

Lesson 2 of 7 · AI Production Engineering

AI Gateways and Model Routing

Key idea

A gateway centralizes credentials, attribution, limits and routing, and it makes one component critical to everything. Route to the cheapest model that passes your evals, and measure cost per completed task, because a cheap model that fails can cost more. And don't let a gateway strip what it doesn't understand, since APIs change monthly.

> *Before: retries, model tiers, HLD API gateways. Teaches: what an LLM gateway does differently, pass-through rules, build or buy, routing strategies, and cost per task. Used by: L4, L5, L6, and L7, which designs one.*

The hook: three small problems

An invented team, with a common situation:

Problem	Root cause
A provider key leaked from a developer's laptop	Everyone holds the real key
Spend jumped, and nobody can say which feature did it	No attribution by team or feature
Moving to a new model takes three teams two weeks	The model ID is hard-coded in 40 places

Each could be fixed separately, in every service. All three have the same cheap solution: put one component on the path to the model.

Intuition: the company card desk

Picture a firm where every employee has the company's master credit card. Some spending would never be traced, and if one card is stolen, everyone needs a new one.

So the firm gets a card desk. Staff ask for what they need, and the desk holds the real card. It knows who bought what, enforces budgets, and can switch suppliers without telling anyone. The cost is that when the desk closes for lunch, nobody can buy anything, and the desk has to keep up with every new kind of purchase.

Mechanism

What an LLM gateway adds

HLD M04 taught API gateways. Five things differ here:

Concern	API gateway	LLM gateway
Limit unit	Requests	Tokens and dollars, known only after the response
Response	Short	Long-lived stream that must not be buffered
Routing	By path	By model, cost, capability, and quality
Retry	Same result	A different model gives a different answer
Cache	Exact URL	Prefix caching, or meaning (L5)

Anthropic's Claude Code docs list what a gateway provides, and it's a fair summary: credentials stay server-side, usage is attributed, budgets and limits are enforced in one place, requests are audit-logged, and the provider can change without touching developer machines. They also state the trade-off: it becomes infrastructure you operate.

Anthropic now also has a **Claude apps gateway**, a self-hosted gateway with SSO sign-in and OTLP telemetry, for Claude Code. For other products, the same docs say that Anthropic doesn't endorse, maintain or audit third-party gateway products.

Where it sits

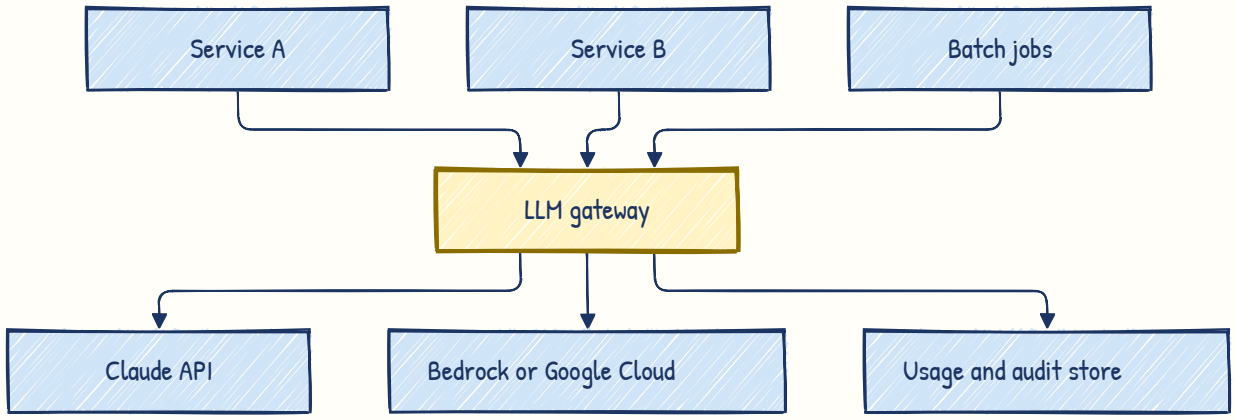


Figure 1. Services hold gateway credentials only. The gateway holds the real ones, and is on every path to a model.

The request's life

Step	What happens	Note
Authenticate	Gateway key maps to team and tenant	Revoking one person is easy
Check limits	Rate and budget	Costs are unknown yet, so reserve (L7)
Route	Select model and target	Policy, not caller's guess
Forward	Add the real credential	Pass through the rest
Relay	Stream each event as it arrives	**Never buffer**
Meter	Read usage from the stream	Settle against the reservation
Record	Metrics and audit	Asynchronously

Pass-through: the trap that's unique to LLM gateways

Claude Code's gateway compatibility guide is the best public document on this, as it says what actually breaks:

Rule from the guide	Why
Forward `anthropic-version` and `anthropic-beta` unchanged	The set changes with releases
Forward beta headers verbatim, and **don't allowlist individual values**	An allowlist blocks each new feature, silently
Don't buffer a stream	Buffering makes Claude Code stall
Deliver the full event sequence, through `message_delta` and `message_stop`	The client relies on it to know that a response is complete
Keep `ping` events	They may be the only traffic during a long pause
Return `retry-after` as integer seconds	The client reads it to decide when to retry
Pass `x-should-retry` through	It's an input to the retry decision
Forward error bodies unmodified	Clients use them to react to a rejected capability

Real timings from my small gateway (L7), on a laptop with a local fake:

```
20 events, 10 ms apart, 30 runs, median ms, one laptop:
  direct:         first text  20, whole stream 240
  through relay:  first text  26, whole stream 249
  buffering gateway: first text 247
```

The relay costs a few milliseconds. The buffering gateway's first text arrives when the whole answer is finished. **This is local loopback with a fake, so it measures buffering's shape, not production overhead.** Real overhead includes network hops, TLS, auth lookups, and logging. Measure yours.

A gateway that parses and re-serializes requests will eventually break a feature it's never heard of. Treat the body as opaque where you can, and read only what you need.

Build, buy or neither

Option	When	Watch for
Nothing, plus a client library	One team, one provider, few services	Attribution and key rotation, done by hand
Provider's tools	Workspaces cover your limits and reporting	Coarser, and one provider only
Open source or vendor gateway	Several teams, standard needs	Keeping up with API changes, and its security. It sees all prompts
Build your own	Unusual routing or strict data rules	You now own a critical system, plus API changes

Workspaces deserve a look first. They give per-workspace limits and spend caps, and the Usage and Cost Admin API reports by key and workspace. That may be enough for a small organization.

I haven't evaluated any gateway product for this course, so I can't recommend one.

Routing strategies

"Which model?" is usually answered once, and then never revisited. Here are the options, from simple to complex:

Strategy	How	Good for	Risk
Fixed per feature	Config: this feature uses this model	Most systems	Forgets to re-check
By task type	Classifier or the caller's label picks a tier	Mixed workloads	Misrouting
By customer plan	Paid gets the larger model	Pricing tiers	Quality differences become visible
Escalate on failure	Try small, then larger if validation fails	Checkable outputs	Added latency on failures
By cost budget	Near the budget, drop a tier	Hard ceilings	Quality falls silently
Automated per request	A model chooses the model	Rarely justified	Complex to evaluate and debug

Escalation is often the best of these, when you can check the result in code, using schema validation (TS 4.4) or a test. Failures cost a small retry, and most requests stay cheap.

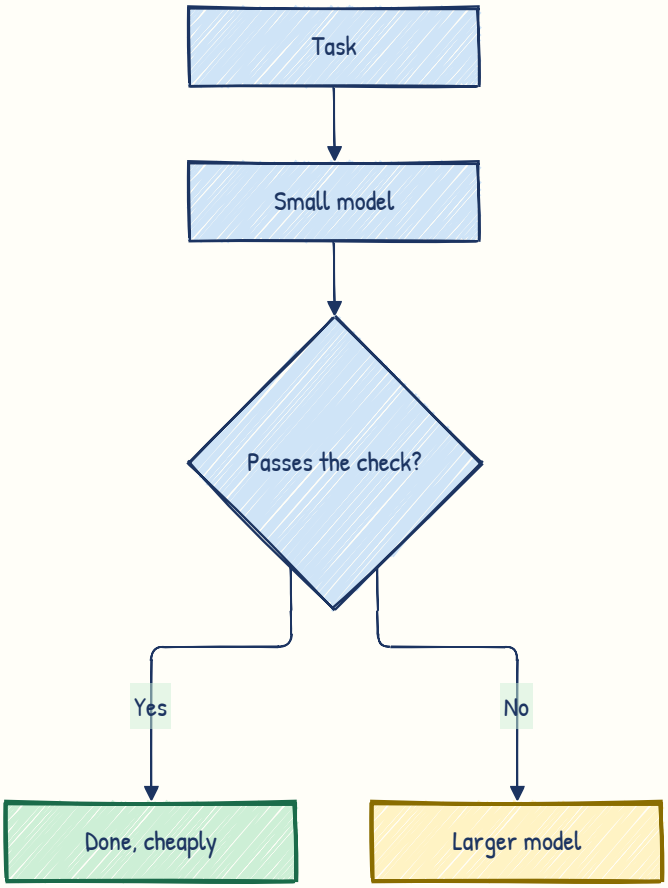


Figure 2. Escalate only when a check fails. If the output can't be checked, this doesn't apply.

Cost per completed task

This is the number to optimize. Prices are from the pricing page today. **Success rates and rework cost are illustrative inputs.** Real output:

```
Cost per completed task, illustrative success rates:
  claude-haiku-5-5   success  70%  attempt $0.0007  model/task $0.0009  with $0.50 rework $0.1509
  claude-sonnet-5-5  success  92%  attempt $0.0130  model/task $0.0141  with $0.50 rework $0.0541
  claude-opus-5-5    success  97%  attempt $0.0260  model/task $0.0268  with $0.50 rework $0.0418
```

Read this in two directions:

→ **Model cost alone says Haiku wins easily,** at about 30 times cheaper than Opus per task.

→ **Add $0.50 of human rework per failure, and it reverses.** Opus has the lowest total under these inputs.

With a rework cost of zero, as when a code check catches failures and retrying is free, cheap wins. **The cheapest model depends on what a failure costs you, so measure both.** AI02 L7 introduced the idea, and this is what it looks like in numbers.

And routing can pay well when the mix suits it. With the same request size, and an invented 60/30/10 split:

```
Routing mix, illustrative shares, per million requests:
  all Opus 5.5: $26,000. Routed 60/30/10: $6,890 (74% lower)
```

Prices at the time of writing, per million tokens:

Model	Input	5-minute cache write	Cache read	Output
Haiku 5.5, prompts up to 100,000 tokens	$0.10	$0.125	$0.01	$0.50
Sonnet 5.5	$2.00	$2.50	$0.10	$10.00
Opus 5.5	$4.00	$5.00	$0.20	$20.00

Haiku 5.5 is priced by prompt length: prompts over 100,000 tokens cost $0.50 and $2.50. That's a cliff for a router to know about. **Check these on the pricing page, as they change.**

Model routing needs evals

A routing rule claims that a cheaper model is good enough. Only evals (L3) can back that up, and only on your task. Keep this record per route:

Field	Example
Task	Ticket classification
Model	`claude-haiku-5-5`
Eval result, and date	96.1% on 800 tasks, 2026-10-14 (invented)
Fallback	`claude-sonnet-5-5`, also evaluated
Parameters per model	Effort, thinking and tools differ by model
Owner	Named person

Model parameters are part of the route. AI03 L4 covered this, and the errors page lists many model-specific 400s. A router that swaps only the model ID will break.

Exact model IDs, in one place

The hook's third problem has a cheap fix: use a logical name such as `support-classifier`, which config maps to a real ID. Then a model migration is a config change, plus an eval run, plus a canary (L6).

Why the obvious fixes fall short

Obvious idea	Why it falls short
Share one key, but tell people to be careful	No attribution, no revocation
An ordinary API gateway	Buffers, or limits only by request
Parse and validate all requests in the gateway	New API features break
Always use the best model	Costly for easy tasks
Always use the cheapest	Rework can cost far more
A model that picks the model	Hard to evaluate, and it adds latency and cost
One gateway, with no redundancy	A new single point of failure

How Anthropic approaches it

Mechanism	What's published
Gateway guidance	What a gateway provides, plus a compatibility guide for Claude Code
Own gateway	Claude apps gateway for Claude Code, self-hosted
Organization controls	Workspaces with limits, and the Admin API
Model choice	Docs recommend effort and model selection (AI03 L4)
Cost guidance	Cost optimization guidance puts caching first and model selection last

That last row is interesting. The bundled API reference's cost guide orders its levers as caching, input tokens, loop hygiene, output tokens, batch, then effort and budgets, and only then model selection. It calls those last ones trade-offs, as they can affect quality, where caching doesn't. L5 follows this order.

The production architecture

This is a recommended reference design, not a specific company's system. L7 designs this in depth.

Baseline: services call the provider directly.

Improved: one gateway, with central keys and metering. New problem: it's a single point of failure, and it's on the hot path for every feature.

Production: stateless gateway replicas with config-driven routes, which degrade rather than fail.

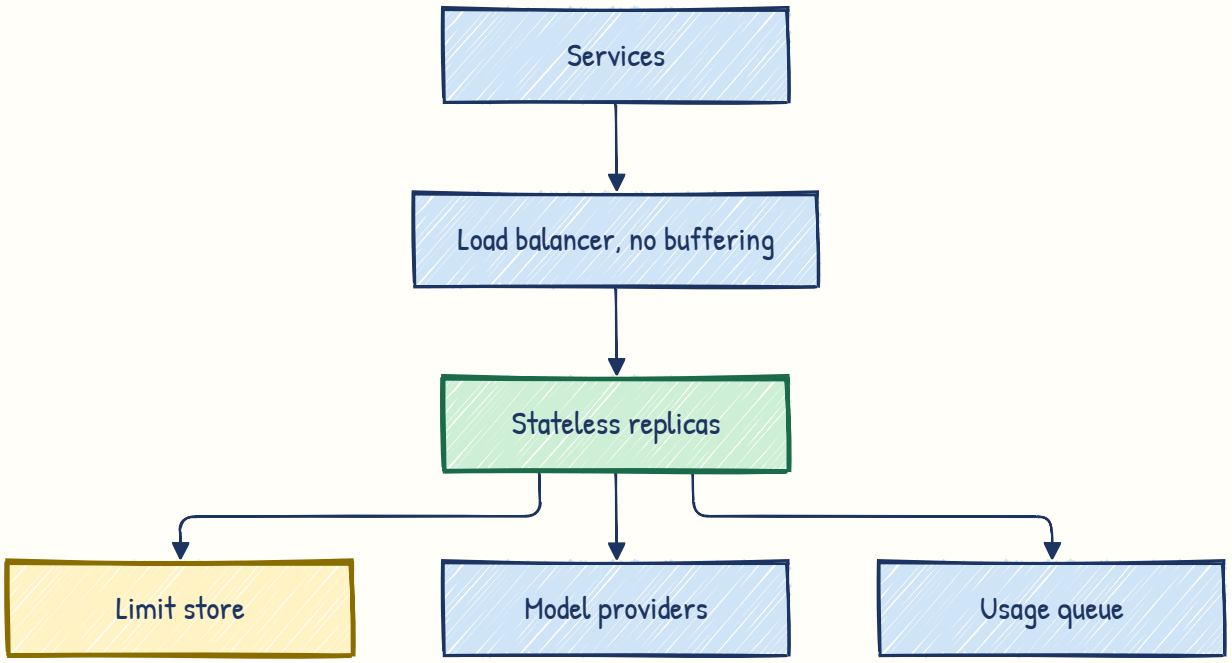


Figure 3. Replicas are interchangeable, because state lives elsewhere. A failed replica costs only the streams it was carrying.

Component	Why it exists	How it communicates
Stateless replicas	Scale and survive failures	Load balancer
Route config	Logical name to model, parameters and fallback	Versioned, cached locally
Limit store	Budget and rate state, across replicas	Redis or similar
Usage pipeline	Attribution and audit, off the hot path	Queue
Credential store	Real keys, rotated	Secret manager
Eval link	A route change needs a passing eval	CI

Failures:

Failure	How you'd notice	What to do
Gateway outage	All AI features fail	Replicas, and maybe a controlled direct path for critical services
Limit store down	Limit errors	Decide in advance. Fail open with a local cap, or fail closed
Stream buffered by a proxy in the path	Time to first text jumps	Check every hop, and alert on that metric
A beta feature breaks	Client errors after an API release	Forward unknown headers. Test the newest SDK
Misrouted task	Quality falls on one route	Per-route evals and dashboards
Gateway holds all prompts	Security review	Treat as a sensitive store (AI09 L4)

Java

The relay's central decision is flushing at each event boundary:

```
out.write((line + "\n").getBytes(StandardCharsets.UTF_8));
if (line.isEmpty()) {
    out.flush();
}
```

A blank line ends an SSE event. Flushing there keeps the stream live, without flushing every line. L7 has the whole gateway.

The scripts stay simple, and the cost calculation is one small method, with cache reads priced separately:

```
return (input * price.input() + cacheWrite * price.write5m()
        + cacheRead * price.read() + output * price.output()) / 1e6;
```

Trade-offs

Decision	Benefit	Cost
Gateway	Central control	Critical new component, added hop
Library instead	No new hop	Each language needs one. Weak on revocation
Opaque pass-through	New features work	Less validation
Per-request routing	Lower cost	More to evaluate
Fixed routes	Simple and predictable	Possibly overpaying
Second provider	Independent failures	Double the testing

When not to build a gateway: one team, one provider, and few services. Use a small client library plus workspaces, and revisit when a second team arrives.

Common mistakes

Allowlisting beta headers. It feels secure, and it silently disables features with every release.

Common mistakes

Comparing models on token price alone. The metric is cost per completed task, including failures, retries and rework.

What to say in an interview

"A gateway centralizes credentials, attribution, limits and routing. Its special needs are token-based limits known only after the response, streams that mustn't be buffered, and opaque pass-through, so new API features keep working. I'd route to the cheapest model that passes evals, and I'd measure cost per completed task. And since it's now critical, it's stateless, replicated, and has a decided behaviour for when its limit store fails."

Certification lens

How this lesson maps to CCAR-F

Gateways and routing aren't in the guide. Exam relevance is indirect: using a smaller model for a simple subtask is a recurring idea in agent questions. AI12 returns to gateways through Claude Code's settings.

These are original practice questions, not real or leaked.

Q Q1 · A platform team proposes an allowlist of permitted `anthropic-beta` values in the gateway, for safety. Respond.

A Claude Code's compatibility guide says to forward that header verbatim and not to allowlist values, as the set changes with releases. An allowlist breaks new features, quietly. If there's a real policy need to block a capability, block that specific one, and keep unknown values passing.

Q Q2 · Model A costs $0.002 per attempt at 80% success, and model B costs $0.01 at 98%. Failures are caught by a code check, and retried for free. Then consider failures that reach a person at $2 each. Which wins in each case?

A Retrying until success costs A $0.0025 per task, and B about $0.0102, so A wins. If each failure instead reaches a person: A costs $0.002 + 0.2 × $2 = $0.402, and B costs $0.01 + 0.02 × $2 = $0.05, so B wins. The cost of a failure decides it.

Q Q3 · What should a gateway do when its limit store is unreachable?

A Decide this before it happens. Failing closed stops all AI features because of a counter. Failing open risks overspending. A common middle path is a conservative local cap on each replica, plus an alert, with fail-closed for the riskiest routes. Whichever you pick, test it.

Revision - Lesson 2

→ A gateway gives credentials, attribution, limits, audit and provider switching, and it's now critical.

→ It differs from an API gateway: token limits known late, streams, routing by model, and pass-through.

→ Forward beta headers verbatim. Don't buffer. Keep the full event sequence and pings. Forward error bodies.

→ Check workspaces and a client library before building anything.

→ Strategies: fixed, by task, by plan, escalate on failure, by budget, automated per request.

→ Escalation works well when output can be checked in code.

→ Measure cost per completed task. Failure cost can reverse the ranking.

→ Haiku 5.5 has a price step at 100,000 tokens. Check prices on the page.

→ Use logical model names, with parameters per model and a fallback that's been evaluated.

Your notes